

Argus × n8n — UX

Surfaces, states, and the palette — companion to the automations plan

Design proposal · 1 August 2026 · Project Argus · brainstormed with Ethan

1. What this adds

The automations plan settled architecture: an n8n instance registered by URL and API key, workflows tagged `argus` discovered over n8n's REST API, an inbound `/api/external/*` endpoint on its own port behind a tunnel, and widgets whose payloads declare their own renderer. It did not settle where any of it appears or how it behaves when things go wrong. This document does, grounded in five pieces of existing software that already solved parts of the problem.

It also corrects two factual errors in the plan (§3) found while reading n8n's documentation properly.

2. Precedents

SOFTWARE	WHAT IT ALREADY SOLVED	WHAT WE TAKE
Home Assistant	Entities push state; cards render by type; <code>unavailable</code> / <code>unknown</code> are first-class states, not edge cases. Dashboards auto-generate until you "take control". Button cards carry confirmation as a configurable object with custom text, plus a lock for sensitive controls.	The dashboard placement model (§5), the principle that a card must stay useful the day its source fails, and confirmation-as-object rather than boolean (§9).
Raycast	Keyboard-first palette; command arguments rendered as inline fields; list and detail views; forms for content creation. Documented guideline: required arguments before optional ones.	The whole action-invocation model (§7), field ordering, and the detail view as a place for results.
Grafana	"No data" silently breaking dashboards; the explicit distinction between a null result rendering <code>N/A</code> and a genuinely empty query.	The four-state widget model (§6) — the plan previously collapsed "empty" and "never arrived" into one blank panel.
Apple Shortcuts	Widget buttons showing a progress indicator as actions advance, with cancel; pausing to ask when an action needs data it does not have.	Run progress and cancellation (§9), rather than a button that spins with no way out.
n8n Form Trigger	12 typed field elements, per-field required/placeholder/default, a <i>Respond When</i> mode, and Basic Auth on the trigger.	The form schema itself (§8) — and the corrections below.

3. Corrections to the plan document

§5.2 UNDERSTATED THE FIELD TYPES

The plan listed seven. There are **twelve**: Checkboxes, Custom HTML, Date, Dropdown, Email, File, Hidden Field, Number, Password, Radio Buttons, Text, Textarea. Three of them raise questions the plan never asked, resolved in §8.

§5.4 DESCRIBED A SUPPORTED FEATURE AS A WORKAROUND

The Form Trigger has its own *Respond When: Form Is Submitted | Workflow Finishes* setting, and on *Workflow Finishes* n8n displays the workflow's error if it fails. The result-return path the plan invented already exists as a first-class option. The *Immediately*-default warning still stands — it is simply a documented setting to change rather than a trick.

The trigger also supports **Basic Auth**, which Argus must handle when firing a form URL directly. The plan did not account for this at all.

4. The surface model

Automations split cleanly into two kinds that want opposite treatment, so they get different homes rather than one compromised page.

KIND	HOME	WHY
Displays	The dashboard	Passive and glanceable. A weather widget you have to summon is pointless — it earns its keep by being visible without being asked for.
Actions	The command palette	Invoked, numerous, and long-tailed. Scales to hundreds without consuming screen space, and matches the keyboard-first affordance Argus already has.
<code>/automations</code> — management only		Install, inspect health, fix. Not a place you visit to <i>use</i> an automation.

5. The dashboard

5.1 Auto-place, then take control

A new widget appends to the dashboard automatically, so an automation is useful the moment it first pushes — no configuration step between installing and benefiting. The first time the layout is rearranged or a widget removed, Argus stops auto-placing and the arrangement becomes the user's. This is Home Assistant's model and it is proven: zero-config by default, full control the moment it is wanted, and no mode switch to discover.

5.2 Automation-fed panels are marked

An `AUTO` tag beside the eyebrow and a mono freshness line underneath. Native panels get neither.



Left: a native panel. Right: the same panel language, plus provenance and freshness.

This matters more after the migration than before it. Once Google Calendar and Todoist are n8n workflows, the panels you rely on most are the ones whose data depends on a second service staying alive. Provenance stops being trivia at that point.

6. Widget states — three of four are not failures

Grafana had to make this distinction explicit and so must Argus: a panel showing nothing because the source legitimately returned nothing is a different thing from a panel showing nothing because nothing ever arrived. The plan collapsed them into one blank card.

STATE	RENDERING	MEANING
LIVE	Normal panel, <code>updated 3m ago</code>	Fresh within its expected interval.
STALE	Dimmed, amber eyebrow, <code>STALE 4h</code> badge, <i>last good data still shown</i> , true age and expected cadence	Past 2.5× the expected interval. Showing old data labelled as old beats showing nothing.
EMPTY	Normal panel, plain sentence — "Nothing scheduled today." — normal freshness line	Pushed successfully, genuinely zero items. Must read as reassurance, not breakage.
WAITING	Faint eyebrow, <code>WAITING</code> badge, "No data received yet", an OPEN IN N8N action, and how long it has been	Installed, nothing has arrived yet.

CALENDAR

STALE 4H

09:00 Standup30m

14:00 Advising1h

last push 04:12 · expected every 15m

CALENDAR

WAITING

No data received yet.

OPEN IN N8N →

installed 12m ago · workflow may not be active

STALE keeps the last good data; WAITING offers the fix rather than just naming the problem.

Why WAITING links to n8n: the two overwhelmingly likely causes are a workflow that was never activated and a credential that was never granted. Both are fixed in n8n, not in Argus, so the state that names the problem should also open the place it gets solved.

7. The command palette

`web/components/CommandPalette.tsx` already exists and already proves the two patterns needed: the `SEARCH` action switches the panel into a different mode with its own input behaviour and result list, and the `reindex` action demonstrates toast → poll → toast-the-real-outcome for long work. This is extension, not new construction.

7.1 Three modes

1. **Filter** (exists) — automations appear as a new `AUTO` kind in the existing `KIND · label · hint` rows, hint showing `3 fields` or `confirm`.
2. **Form** (new) — selecting an automation with a Form Trigger switches the panel into its fields. Required before optional, per Raycast's guideline. Tab moves between fields, Enter runs, Escape returns to the list.
3. **Detail** (new) — a returned `widget` renders inline in the panel. Escape closes; nothing is persisted unless explicitly pinned, so a one-off lookup never clutters the dashboard.

`ack` and `status` results stay toasts — the existing pattern handles them correctly and a green tick does not deserve a panel.

7.2 The refactor this requires

`PALETTE_ACTIONS` is a static module-level array consumed directly in render. Automations are fetched, so it becomes static-plus-dynamic: the constant list merged with registered automations at render time. The module's stated principle — a plain array, no command-framework dependency — is preserved.

8. Rendering all twelve field types

Argus renders every Form Trigger field type natively. Three needed decisions the plan never made.

FIELD	RESOLUTION
File	Multipart upload with its own 10 MB ceiling, separate from the 256 KB JSON body cap. That cap exists to stop a public endpoint filling the disk with widget payloads; the reasoning does not transfer to a user-initiated upload.
Password	Rendered masked. Held in memory for the duration of the request only — never persisted, never logged, never written to run history , and cleared from the form on submit. This does put Argus in the secret-relay business, which it was not in before.
Custom HTML	Sanitised and rendered. n8n already forbids <code><script></code> , <code><style></code> and <code><input></code> inside these fields, so Argus applies that same allowlist plus strips event-handler attributes — one sanitiser shared with the <code>text</code> widget's markdown.
Hidden Field	Not rendered; its default or query-parameter value is passed through unchanged.
Checkboxes / Radio / Dropdown	Native controls from <code>fieldOptions.values[]</code> ; Checkboxes honour the selection-limit setting.
Date / Email / Number / Text / Textarea	Direct mappings, with placeholder and default applied where n8n supports them.

A CONTRADICTION RESOLVED RATHER THAN INHERITED

The plan rejected "workflow returns markup" as a widget contract on XSS grounds, then this document accepts Custom HTML fields — the same risk through a different door. The resolution above is what makes both positions consistent: markup is never trusted, only ever rendered through one shared sanitiser with an allowlist n8n itself already enforces.

9. Action feedback and safety

- **Progress, not a spinner.** A running action shows advancement and remains cancellable, per Shortcuts. Cancel calls n8n's execution-stop API — the run is genuinely stopped, not merely un-watched.
- **Confirmation is an object, not a boolean.** Following Home Assistant: `argus:confirm` gates the fire, and the workflow can supply the prompt text so it names the actual consequence rather than asking "are you sure?".
- **In-flight locking.** The control disables while a run is open and each fire carries a run id, so a double press cannot send two emails.
- **Failures are recoverable.** The HUD line reports the most recent failure, but it moves on — the ACTIVITY tab is where a failure remains findable afterwards.

10. `/automations`

Three tabs. **GALLERY** installs bundled templates. **ACTIVE** lists every registered automation with what it is and its real state — `display · timeline · pushed 3m ago · every 15m` against a LIVE / STALE / WAITING badge, or `action · 3 fields · last run 12:04 ok` with RUN. Failures are visible in the list rather than buried.

ACTIVITY is an audit trail — every run, push, capture and failure with a timestamp, filterable by automation. It was originally specified as a Notion-style database view over captured data, which was wrong: captures are written through `writer.py` into the vault, where the existing activity feed, notes and search already surface them. Rebuilding that would have duplicated Argus. What is genuinely missing is a record of what the automations *did*, which is the question that actually gets asked when something looks wrong.

11. Conventions this UI must not break

- Type scale tokens only — `text-label`, never `text-[13px]`. Every token is rem so the reader's own browser font size reaches the app.
- `Panel.tsx` is the surface; the `|` eyebrow glyph comes from it, not from a class.
- `Dialog` is the only overlay implementation; `ConfirmDialog` / `useConfirm` is the only confirmation path. No `window.confirm`.
- Never write `focus:outline-none` — Tailwind emits it at a specificity that beats the global `:focus-visible` rule.
- `.shell` and `lg:grid-cols-shell` are the only width and split definitions.
- The e2e suite is tightly coupled to visible text, uppercase accessible names, the `|` glyph and `<section>` / `<li>` / `<form>` element choices. Classes are free to change; those are not.

12. Open questions

1. **Widget history.** Rejected for the third tab, but "weather over the past month" is a real thing to want. It needs a retention and pruning policy before it can be designed, and has none.
2. **Basic Auth on Form Triggers.** Where those credentials live if a user protects a form — plausibly the same keyring pattern as everything else, but undesigned.
3. **Take-control granularity.** Whether taking control of the dashboard layout is global or per-widget. Global is simpler; per-widget means a new automation can still auto-place after you have arranged the others.
4. **Palette scale.** Argus has no precedent for more than a dozen palette actions. Whether `AUTO` rows need grouping or ranking past ~30 automations is unknown until it happens.

Argus × n8n automations — UX design, 1 August 2026. Companion to the automations plan. Concept approved; not yet an implementation plan.